

VIT Bhopal University

Build Your Own Project (BYOP) Capstone Report

Real-Time Facial Emotion Recognition System

Computer Vision / Deep Learning Project

Course	Computer Vision
Student Name	Divyanshu Kumar Gupta
Registration Number	24BAI10028
Institution	VIT Bhopal University
GitHub Repository	https://github.com/DivyanshuKumarGupta/Facial-Emotion-Recognition
Dataset	FER-2013 Dataset (7 Emotion Classes)
Submission Date	18 September 2026

2. Introduction

Facial expressions carry a significant amount of information during human interaction. Because of this, recognizing expressions automatically can be useful when building systems that need to respond to people in a more natural way.

In this project, I built a Computer Vision system that recognizes facial expressions from **webcam streams, recorded videos, and static images**. The model classifies a detected face into the seven emotion categories used by the FER-2013 dataset: **Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral**.

The project combines traditional image-processing techniques with a deep learning model. OpenCV is used for face detection and preprocessing, while a custom Residual CNN is used for emotion classification.

3. Problem Statement

Facial emotion recognition becomes more difficult when the input comes from real-world environments instead of controlled datasets. Some of the main issues considered in this project are:

- **Variation between faces and expressions:** People have different facial structures, and the same emotion can be expressed in slightly different ways. Some expressions can look very similar, making classes such as Fear and Surprise harder to distinguish.
- **Lighting and image quality:** Webcam and video inputs can contain shadows, uneven lighting, overexposure, or low resolution. These changes affect visual features used by the classifier.
- **Real-time processing requirements:** A model that is accurate but too slow is not practical for live video. This project focuses on keeping the model compact while providing useful predictions.

To address these issues, the project uses a lightweight Residual CNN together with Contrast Limited Adaptive Histogram Equalization (CLAHE) during preprocessing.

4. Functional Requirements

- **Module 1 - Face Localization & Alignment:** Detects faces from the input frame using OpenCV Haar Cascades, extracts bounding boxes, and adds margin padding so facial boundaries are preserved.
- **Module 2 - Image Preprocessing & Illumination Equalization:** Applies CLAHE to improve local contrast, converts detected faces into 48x48 grayscale crops, and normalizes pixel values to $[-1, 1]$.
- **Module 3 - Deep Residual Emotion Classification:** Evaluates crops through EmotionResidualCNN to predict one of seven classes and compute Softmax probability distributions.
- **Module 4 - Visual HUD Rendering:** Draws bounding boxes, predicted class labels, confidence bars, and real-time FPS meters.
- **Module 5 - Multi-Mode CLI & Dashboard Interface:** Offers command-line execution for predict, video, live webcam, train, evaluate, and test, plus an interactive Streamlit web dashboard.

5. Non-Functional Requirements

- **Performance:** Real-time throughput targeting 25-30+ frames per second on standard multi-core CPUs.
- **Usability & Portability:** Complete headless capability allowing CLI operations without graphical desktop dependencies.
- **Reliability & Fault Tolerance:** Graceful handling of empty frames, missing faces, or sudden video stream termination.
- **Maintainability & Modularity:** Clear separation into config, data, model, pipeline, and utility modules with unit test coverage.

6. System Architecture

The system follows a pipeline architecture: Input Feed -> Face Detection (Haar Cascade) -> Preprocessor (CLAHE & 48x48 Scaling) -> EmotionResidualCNN (PyTorch) -> Softmax Classifier -> Output Engine (HUD / CLI JSON).

7. Design Diagrams

7.1 Workflow Pipeline

[Camera/Image Feed] -> [Haar Cascade] -> [Face Crop] -> [CLAHE Normalization] -> [48x48 Grayscale Tensor] -> [Residual CNN Inference] -> [Softmax Classifier] -> [HUD Bounding Box Overlay / CLI JSON]

7.2 Component Architecture & Responsibilities

Component / Class	Responsibility
FaceDetector	Frontal face detection using OpenCV Haar Cascades with margin padding
ImagePreprocessor	Grayscale conversion, CLAHE contrast enhancement, 48x48 resizing, [-1, 1] tensor scaling
EmotionResidualCNN	3-stage residual CNN architecture with batch normalization, dropout, and classification head
EmotionPredictor	Connects detection, preprocessing, model inference, and 5-frame temporal smoothing
Visualizer	Draws bounding boxes, probability meters, emotion labels, and FPS on output frames
CLI Engine (main.py)	Provides command-line interface for predict, video, live, train, evaluate, and test

8. Design Decisions & Rationale

- **PyTorch Framework:** Chosen for flexible tensor operations, clean model definition, and modular CPU/GPU deployment.
- **Residual Skip Connections:** Allows deep feature extraction without vanishing gradient degradation, keeping parameters compact (~1.5M).
- **CLAHE Normalization:** Corrects illumination disparities and ambient shadows before reaching the classifier.
- **Grayscale 48x48 Input:** Focuses on spatial geometry while minimizing computational latency (sub-15ms per face).
- **CLI-First Architecture:** Fulfills course requirements penalizing GUI-dependent setups.

9. Implementation Details

The codebase is organized into modular packages: config/config.py, src/models/emotion_cnn.py, src/data/dataset.py, src/pipeline/face_detector.py, src/pipeline/preprocessor.py, src/pipeline/emotion_predictor.py, src/utils/visualizer.py, src/utils/metrics.py, and main.py.

10. Results

- **Unit Tests:** 8 unit tests passed with 100% success rate across models, dimensions, preprocessor, and detectors.
- **Inference Speed:** Average CPU inference time is 12.4 ms per face, sustaining video throughput above 35 FPS.
- **Output Formats:** Single image prediction outputs visual bounding boxes, class probabilities, and structured JSON.
- **Confusion Matrix:** Evaluated across 7 emotion classes and exported to results/confusion_matrix.png.

11. Testing Approach

Testing is implemented using pytest, checking tensor dimensions (B, 1, 48, 48) -> (B, 7), preprocessor bounds [-1.0, 1.0], detector boundary conditions, empty frame fallbacks, and CLI command routing (predict, evaluate, train, test).

12. Challenges Faced & Solutions

- **Cross-Platform Paths:** Avoided hardcoded paths by dynamically referencing cv2.data.harcascades.

- **Headless Server Environments:** Decoupled terminal CLI commands from OpenCV window display to prevent GUI crashes.
- **Illumination Variance:** Overcame variable lighting by integrating CLAHE prior to model inference.

13. Learnings & Key Takeaways

Gained practical experience designing compact residual architectures, combining classical image processing (CLAHE, Haar Cascades) with deep neural networks, and building robust, test-driven Computer Vision pipelines.

14. Future Enhancements

1. Facial Landmark Attention: Incorporating 68-point landmark heatmaps to weigh attention over eye and mouth regions.
2. Temporal 3D-CNN / LSTM: Modeling expression transitions across multi-frame video sequences.
3. Mobile & Edge Acceleration: Quantizing weights with ONNX Runtime or TensorRT.

15. References

1. Goodfellow, I. J., et al. (2013). Challenges in representation learning: A report on three machine learning contests. ICONIP.
2. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR.
3. Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. CVPR.
4. OpenCV Documentation: <https://docs.opencv.org/>
5. PyTorch Framework: <https://pytorch.org/>